SwapThread — Circular Fashion & Clothing Swap Marketplace

Internship Project Report & Technical Documentation

Project Name: SwapThread

Project Track: Full-Stack Web Development Internship

Author / Intern: Khushi Sharma

Version: 1.0.0 (Production Release)

Date: October 2026

Repository: SwapThread Marketplace Monorepo

---

1. Executive Summary

Fast fashion has created unprecedented global textile waste, with millions of wearable garments discarded annually while consumers continue to purchase new clothes for single occasions. **SwapThread** is an open, peer-to-peer barter marketplace engineered to eradicate clothing waste by enabling direct, zero-payment exchanges of pre-loved apparel.

Unlike traditional resale marketplaces that impose commission fees, seller margins, and payment gateway deductions, SwapThread operates on a pure barter model. To overcome the classic challenge of traditional barter—the "coincidence of wants" and subjective value estimation—SwapThread introduces:

1. A **deterministic swap value algorithm** that calculates fair exchange benchmarks from brand tiers, garment categories, and condition ratings.
2. An **in-feed negotiation chat** that allows traders to coordinate meetup or courier details securely.
3. **Geospatial radius filtering** across major Indian metropolitan areas to encourage local handoffs and minimize shipping emissions.
4. A **comprehensive admin panel** for dispute resolution, platform governance, and moderation.

2. Problem Statement & Project Objectives

2.1 The Problem

- **Textile Waste & Fast Fashion:** The average garment is worn fewer than 7 times before being discarded.
- **Resale Friction:** Platforms like eBay, Poshmark, or Depop require payment gateways, merchant KYC, and fee deductions of 15%–25%.
- **Lack of Dedicated Swap Platforms:** General classifieds (OLX, Craigslist) lack item-for-item negotiation state machines and fair value guarantees.

2.2 Primary Project Objectives

- Build an intuitive, responsive web application for peer-to-peer garment swaps.
- Implement secure authentication with hashed passwords, JWT session cookies, and role separation (Customer vs. Admin).
- Store and serve high-resolution garment imagery using cloud storage (Cloudinary free tier) and MongoDB.
- Ensure 100% production readiness: 0 hardcoded localhost references, full SPA routing fallback, automated seed data, and a live health monitoring endpoint.

---

3. Technology Stack & System Architecture

3.1 Technology Stack

- **Frontend:** React 18.3.1 (Single Page Application)
- **Routing & State:** React Router DOM v6.28.0, Context API (`AuthContext` , `ToastContext` , `MetaContext` , `ThemeContext`)
- **Styling System:** Vanilla CSS Tokens + Bootstrap 5.3.3 utilities with light/dark theme switching
- **Build Tool:** Vite 5.4.11 (ESModules, fast HMR, production minification)
- **Backend Runtime:** Node.js (v20+ LTS)
- **API Framework:** Express.js 4.21.2 REST API
- **Database:** MongoDB Atlas (Production) / Mongoose ODM 8.9.5 with local fallback
- **Cloud Media Storage:** Cloudinary Free Tier (API streaming via Multer memory storage)
- **Security:** Helmet CSP headers, CORS origin enforcement, bcryptjs password hashing, JWT in `httpOnly` secure cookies, express-rate-limit

3.2 System Architecture Diagram

```
```

+---+

Client (React 18 SPA)
- Listings Grid - Item Detail & Q&A - Swap Negotiation
- Value Compare - Member Profile - Admin Panel

+---+

HTTPS / JSON API Requests

v

+---+

Express 4 REST API Server
- Security: Helmet CSP, CORS origin check, Rate Limiters
- Session: httpOnly JWT cookie authentication
- Routes: /api/auth, /api/listings, /api/swaps, /api/admin
- Health Monitoring: /health, /api/health

+---+

/ | \

/ | \

v v v

+------------------+ +--------------------+ +--------------------+

MongoDB Atlas		Cloudinary		Embedded Fallback
- Users		- Upload Stream		- mongodb-memory-
- Listings		- Secure CDN URLs		server (local
- Swaps & Chats		- Zero disk wear		development)

| - Reports | +--------------------+ +--------------------+

+------------------+

```
```

4. Database Schema Design (Mongoose)

4.1 User Schema (`users`)

- `name` : String (2–60 chars, required)
- `email` : String (unique index, lowercase, required)
- `passwordHash` : String (bcrypt, salt factor 10, excluded from standard queries)
- `role` : Enum `['customer', 'admin']` (default: `'customer'`)
- `status` : Enum `['active', 'suspended']`
- `location` : `{ city: String, state: String, area: String, showArea: Boolean }`
- `geo` : GeoJSON Point `{ type: 'Point', coordinates: [lng, lat] }` (2dsphere index)
- `lastActiveAt` : Date

4.2 Listing Schema (`listings`)

- `title` : String (3–80 chars, indexed)
- `description` : String (10–1000 chars)
- `category` : Enum (tops, shirts, jeans, dresses, jackets, ethnic, footwear, sweaters, trousers, activewear, accessories)
- `clothingType` : String (e.g., "Denim jacket", "Kurta")
- `brand` : String (indexed)
- `size` : Enum (XS, S, M, L, XL, XXL, UK sizes, 28–38 waist, Free Size)
- `condition` : Enum `['new_with_tags', 'like_new', 'good', 'fair']`
- `color` : String
- `estimatedValue` : Number (calculated suggested barter value in INR)
- `images` : Array of Image ObjectIds
- `owner` : ObjectId referencing `User` (indexed)
- `status` : Enum `['available', 'reserved', 'swapped', 'removed']`
- `views` : Number (default 0)

4.3 SwapRequest Schema (`swaps`)

- `requester` : ObjectId referencing `User`
- `owner` : ObjectId referencing `User`
- `requestedListing` : ObjectId referencing `Listing`
- `offeredListing` : ObjectId referencing `Listing`
- `status` : Enum `['pending', 'accepted', 'completed', 'rejected', 'cancelled']`
- `exchangeMethod` : Enum `['meetup', 'courier', 'undecided']`

- `message` : String
- `history` : Array of status change events `{ status, by, note, at }`

4.4 Conversation & Message Schema (`conversations` , `messages`)

- Direct negotiation thread linked 1-to-1 with an active `SwapRequest` .
- Support for participant chat messages and automated system transition events (e.g., "Swap accepted").

---

5. Core Modules & Implementation Details

5.1 Deterministic Swap Value Calculator

Rather than forcing users to guess prices, SwapThread implements an algorithmic barter valuation:

$$\text{Estimated Value} = \text{Base Category Value} \times \text{Brand Tier Multiplier} \times \text{Condition Factor}$$

- **Brand Tiers:** Luxury (Gucci, Coach: $\times 2.8$), Premium (Levi's, Nike, Zara: $\times 1.7$), High Street (H&M, Uniqlo: $\times 1.2$), Fast Fashion (Roadster, Max: $\times 0.9$).
- **Conditions:** New with Tags (1.0), Like New (0.85), Good (0.65), Fair (0.45).
- **Fair Match Indicator:** Flags trades within $\pm 25\%$ value disparity as "Fair Match🤝 ".

5.2 Location-Based Matching & Indian Cities

Covers major Indian urban hubs (Bengaluru, Mumbai, Delhi, Gurugram, Noida, Pune, Hyderabad, Chennai, Kolkata, Jaipur, Chandigarh, Ahmedabad). Utilizes the Haversine spherical formula:

$$d = 2R \arcsin\left(\sqrt{\sin^2\left(\frac{\Delta\phi}{2}\right) + \cos\phi_1\cos\phi_2\sin^2\left(\frac{\Delta\lambda}{2}\right)}\right)$$

Allows users to filter swaps within 5 km, 15 km, 50 km, or pan-India.

5.3 Cloud Image Processing

- Multer parses image buffers into RAM memory (`multer.memoryStorage()`).
- File magic bytes (hex signatures) are inspected via `sniffImageType` to prevent MIME-spoofing attacks.
- If Cloudinary credentials are provided, images stream directly to Cloudinary CDN; otherwise, binary buffers are stored safely inside MongoDB documents.

5.4 Administrative Governance & Moderation Shield

- Dedicated admin dashboard restricted to users with `role: 'admin'`.
- Real-time KPIs: Available listings, successful swaps, swap conversion rate, active members, dispute queue.
- Moderation actions: suspend malicious users, reject abusive listings, resolve user reports.

---

6. Testing & Quality Assurance Summary

6.1 Test Suites Execution (Jest)

All unit and integration tests run in-band to verify core services:

```

Test Suites: 9 passed, 9 total

Tests: 75 passed, 75 total

Snapshots: 0 total

Time: 21.272 s

```

- `tests/users.test.js`: Profile updates, location changes, public profiles.
- `tests/auth.test.js`: Registration, login validation, token issuance, logout.
- `tests/listings.test.js`: CRUD operations, image attachments, filter queries.
- `tests/swaps.test.js`: Swap state machine, transition integrity, rollback logic.
- `tests/messages.test.js`: Conversation authorization, message delivery.
- `tests/admin.test.js`: Staff RBAC authorization, user suspension, KPI calculations.
- `tests/valueCalculator.test.js`: Mathematical accuracy of valuation algorithms.
- `tests/geo.test.js`: Haversine distance computations and city centroid geocoding.
- `tests/validate.test.js`: Input sanitization, email validation, length constraints.

6.2 End-to-End Production Verification Results

Executed against the running Node.js production server (`http://localhost:5001`):

1. **Health Check (`/health`):** Returned HTTP 200 OK (`status: 'ok', database: 'connected'`).
2. **Registration:** User registered successfully, password hashed via bcrypt.
3. **Login:** Authenticated session cookie issued (`rewear_token`, httpOnly).
4. **Listing Creation:** Uploaded image buffer, computed barter value of ₹2,040.
5. **Browse & Filter:** Filtered garments by category (`ethnic`) and condition (`like_new`).

6. **Location Filter:** Successfully queried listings in Bengaluru.
7. **Value Calculator:** Returned ₹1,870 for Levi's Jacket (`premium` tier).
8. **Swap Request:** User B proposed trade for User A's kurta (`isFairMatch: true`).
9. **Accept Swap:** Status transitioned from `pending` to `accepted` .
10. **Chat Messages:** Sent and retrieved negotiation messages in swap thread.
11. **Admin RBAC:** Customer access rejected (HTTP 403 Forbidden); Admin access granted (HTTP 200 OK, 48 listings loaded).
12. **Logout:** Cleared session cookie; subsequent `/auth/me` request returned HTTP 401 Unauthorized.

---

7. Challenges & Solutions Encountered

Challenge	Cause	Resolution Implemented
Ephemeral Disk Wipes on Render	Free tier cloud hosts discard disk storage upon container restarts and redeployments.	Integrated Cloudinary streaming API with MongoDB binary buffer fallback; zero images stored on local disk.
Cross-Origin Cookie Blockers	Browsers block third-party cookies across mismatched domains.	Configured single-service architecture in `server/src/app.js` where Express serves both API and built React static bundle from `client/dist`.
SPA Route 404s on Refresh	Direct browser navigation to `/listings` or `/admin` bypasses client-side router.	Implemented SPA fallback in Express (`app.get('*', sendFile(index.html))`) and `client/vercel.json` rewrites.
Spam / Low-Quality Ads	External ad scripts inject gambling and adult dating banners.	Built strict DOM mutation guard in `SponsoredCard.jsx` to block scam domains and replaced with curated eco-fashion partners.

---

8. Conclusion & Future Roadmap

SwapThread proves that digital barter can be modern, fair, and scalable without involving money. By combining deterministic value estimates, local city matching, and real-time negotiation, the platform removes the friction from wardrobe swapping.

Future Roadmap:

1. **Direct Courier Integration:** API integration with Shiprocket / Delhivery for automated tracking slips.
2. **Community Swap Events:** Offline pop-up swap registration with QR-code garment check-ins.
3. **Wardrobe Analytics:** Carbon and water savings tracker for individual users based on garments kept out of landfills.